whattheflip

INF2004, group SE06 (lab P1). µT-Kernel 3.0 everywhere: whattheflip and iris run `third_party/esp32c6-mtk3`, hestia and daphne run `third_party/mtk3smp-rp2040` with `SMP=1`. Last updated 2026-10-20. The frozen week 6 report with the full test and traceability tables sits in `reports/design-review`.

What it does

Sniffs UART and I2C, captures Wi-Fi 6 and 802.15.4 over the air, injects small controlled faults into the hestia and daphne buses, writes PCAP to microSD and streams numbers to a web dashboard.

The hardware

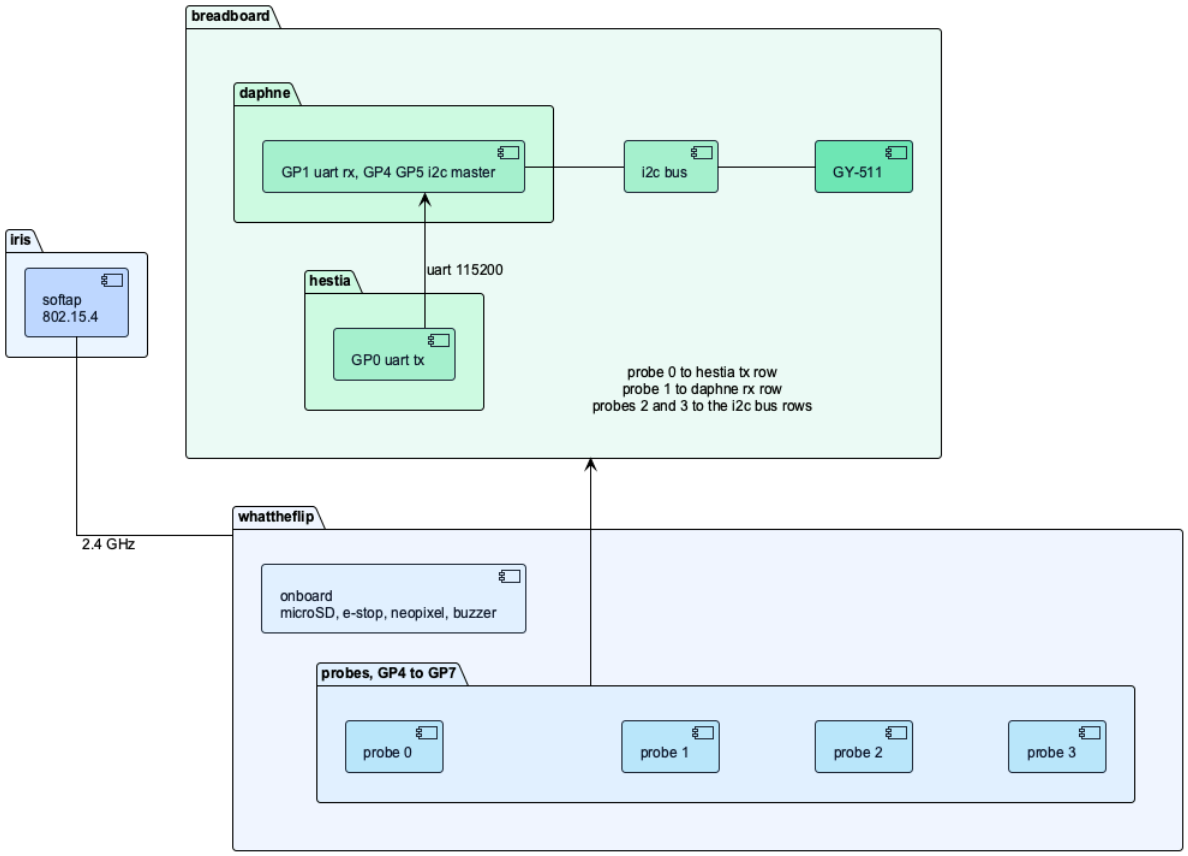
whattheflip is a Waveshare ESP32-C6 module on a Cytron Maker Pi Base. Four probe headers GP4 to GP7 (GPIO4 to GPIO7, probe 0 on GP4). Pinout and wiring reference: `reports/diagrams/ESP32_pinout.png`. On the board: microSD slot on SPI (GPIO18/19/20, EXIO7 for CS), the boot button as e-stop (GPIO9), a neopixel (GPIO8) and a buzzer (EXIO3).

hestia transmits UART on GP0.
daphne listens on GP1 (uart rx) and polls the GY-511 as i2c master at 400 kHz on GP4/GP5. The sensor is wired to daphne's rows only.
hestia and daphne sit on Cytron Maker Pi Base carriers (reset buttons, per-pin leds), jumpered to the shared breadboard with the GY-511.
The four probes plug into breadboard tie rows, sharing 3.3 V and ground.

Probe	Plugs into	Wire
probe 0	hestia GP0 row (uart tx)	220 Ω series, sniff + glitch
probe 1	daphne GP1 row (uart rx)	sniff
probe 2	i2c sda row	sniff, ack kill
probe 3	i2c scl row	sniff, stretch

iris is a bare ESP32-C6. It runs an isolated SoftAP on channel 6 (bssid 02:00:00:00:00:03) and raw 802.15.4 bursts on channel 15.

The frozen design package names these "Node 1/2/3" and "victim A/B": whattheflip is node 1, hestia is victim A, daphne is victim B, iris is node 3.



Who owns what

Person	Subsystems
Tan Yan Qi	auto discovery
Annie Goh	fault injection, hestia, daphne
Zayne Lee	wireless sniffer
Ryan Ke	pcap storage
Joseph Poon	web UI, e-stop, iris

Cuts

Cut	Why
SPI sniffing	the only SPI host is the microSD, and it would eat all four probes. RMT ram also overflows in 4.8 µs at 10 MHz

BLE	no promiscuous api, the 37 hopping channels are not interceptable, and NimBLE eats 45 KB SRAM
OLED	the web dashboard replaces it
level shifters	everything is 3.3 V already

One radio in the C6, so sniffing a remote channel and hosting the dashboard are time-sliced: the dashboard softap and the iris ap both live on channel 6, and the sniffer captures on a target channel in windows of 1 to 2 s before returning.

Requirements

ID	Requirement	Test
FR-1	passive sampling on all 4 pins, 1 µs edge resolution, 1 µA probe leakage	TC-1.1
FR-2	classify uart/i2c within 500 ms, 95 % accuracy, ±2 %	TC-1.2
FR-3	remap any probe to any decoder in 10 ms, no rewiring	TC-1.3
FR-4	capture wi-fi ch 1-13 and 802.15.4 ch 11-26 with rssi and timestamps	TC-2.1, TC-2.2
FR-5	faults bounded: bit flip 10 µs, ack and stretch 50 µs, 5 % duty	TC-3.1, TC-3.2
FR-6	deauth only to the iris bssid, never broadcast	TC-3.3
FR-7	e-stop floats probes in 200 ns, tasks settle within 5 ms	TC-4.1
FR-8	pcap opens clean in wireshark	TC-4.2
FR-9	dashboard updates at 1 to 2 Hz	TC-5.1
FR-10	fault alert reaches the dashboard in 200 ms	TC-5.2

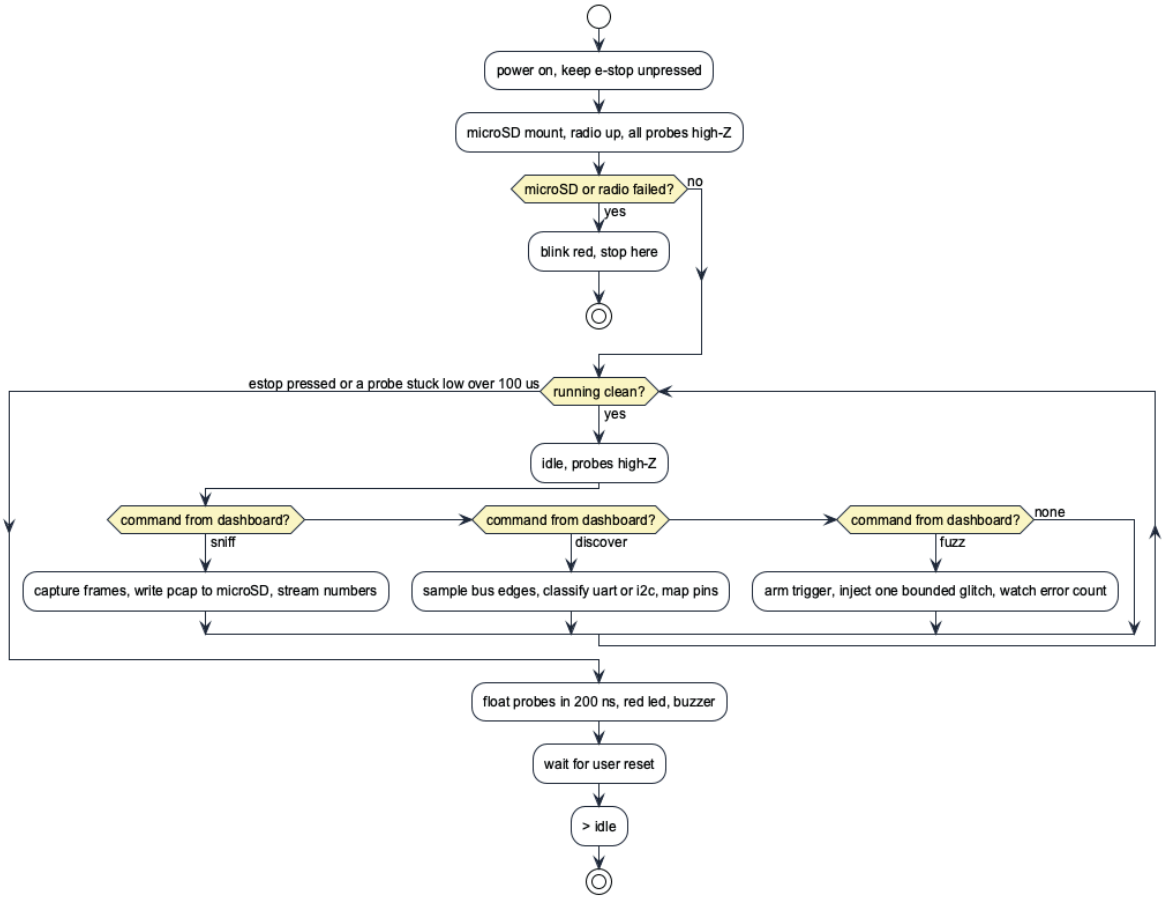
Folder layout

```
whattheflip/
├── whattheflip/      probe, fuzzer, sniffer, storage, web,
safety (µT-Kernel)
├── hestia/           uart tx (µT-Kernel SMP, core 0)
├── daphne/           uart rx, gy-511 i2c master (µT-Kernel SMP)
├── iris/             softap, 802.15.4 tx (µT-Kernel)
└── shared/           code used by more than one board, if it
shows up
```

Tasks

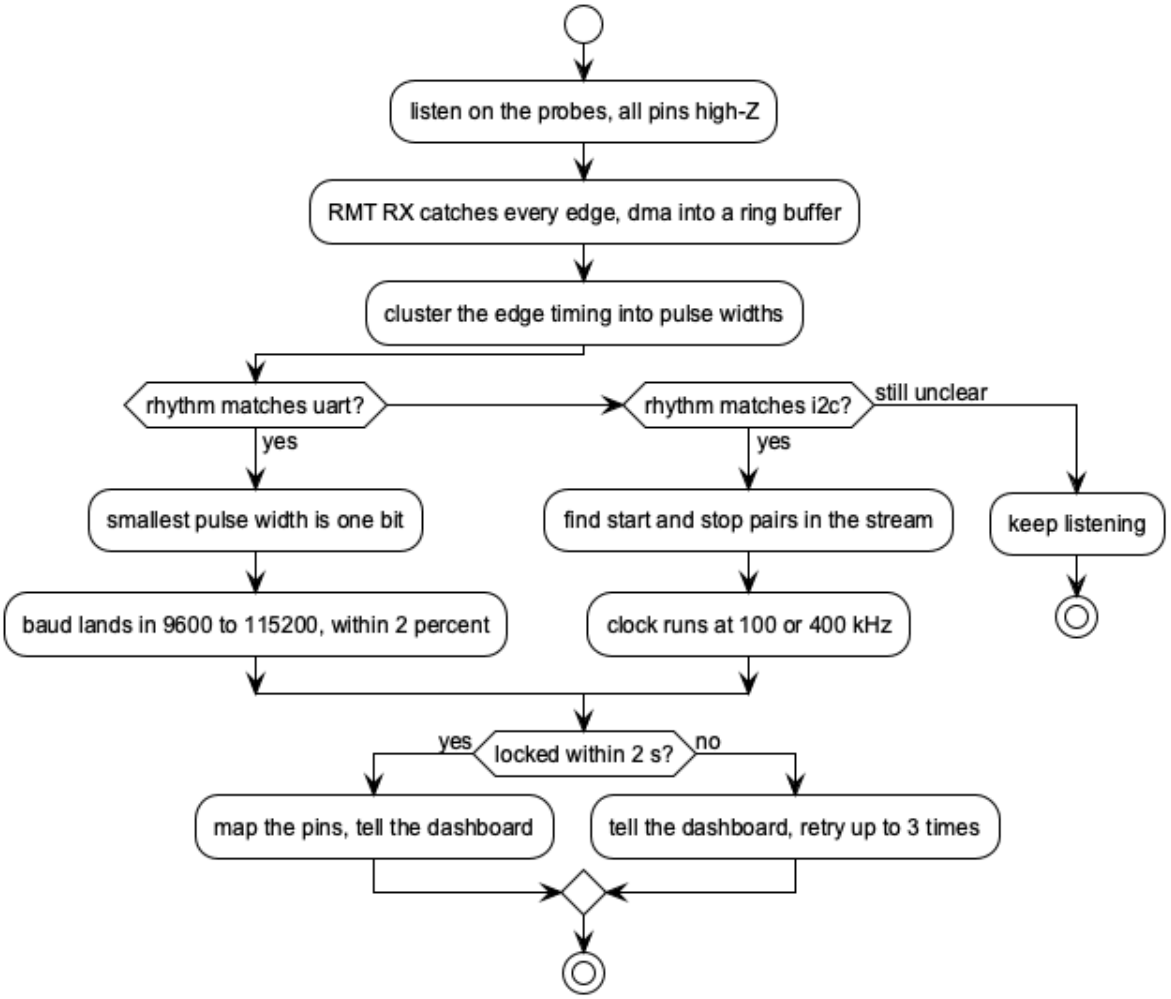
Task	Owns	What it does
TSK_SAFETY	e-stop isr, neopixel, buzzer	floats all probes from the isr in 200 ns, latches the fault state
TSK_WIRELESS	radio	moves frames out of the promiscuous callback, budgeted under 5 µs, into the pool; drops the frame rather than blocking
TSK_FUZZER	ETM, RMT TX	arms the trigger, the hardware times the glitch
TSK_DISCOVERY	RMT RX, probes	dma edge timing into ram, classify and map pins
TSK_STORAGE	microSD	pcap writes and sector flushes off a queue of depth 16
TSK_DASHBOARD	wi-fi socket	json at 1 to 2 Hz off a queue of depth 8

Victim tasks bind to cores with `TA_ASSPRC`: hestia runs the uart tx burst on core 0; daphne runs uart rx with framing checks on core 0 and the gy-511 polling at 400 khz plus its moving-average filter on core 1, held outside the kernel lock; iris runs softap and 802.15.4 bursts. Buffers come from the 32 KB pool `MPF_PACKETS` (16 blocks of 2048 B), taken with a non-blocking `tk_get_mpf` under `TMO_POL` that drops the packet when the pool is full. Queues are `tk_cre_mbf`: `MBF_STORAGE` at depth 16, `MBF_TELEMETRY` at depth 8. Probes are guarded by `MTX_PROBE_ACCESS`. The e-stop isr sets `FLG_SAFETY_ABORT`; discovery sets `FLG_PROTO_LOCKED` above 95 % confidence. Locks are `tk_loc_mtx` with `TA_INHERIT`, other wakes are `tk_set_flg`, periods are `tk_cre_cyc`.

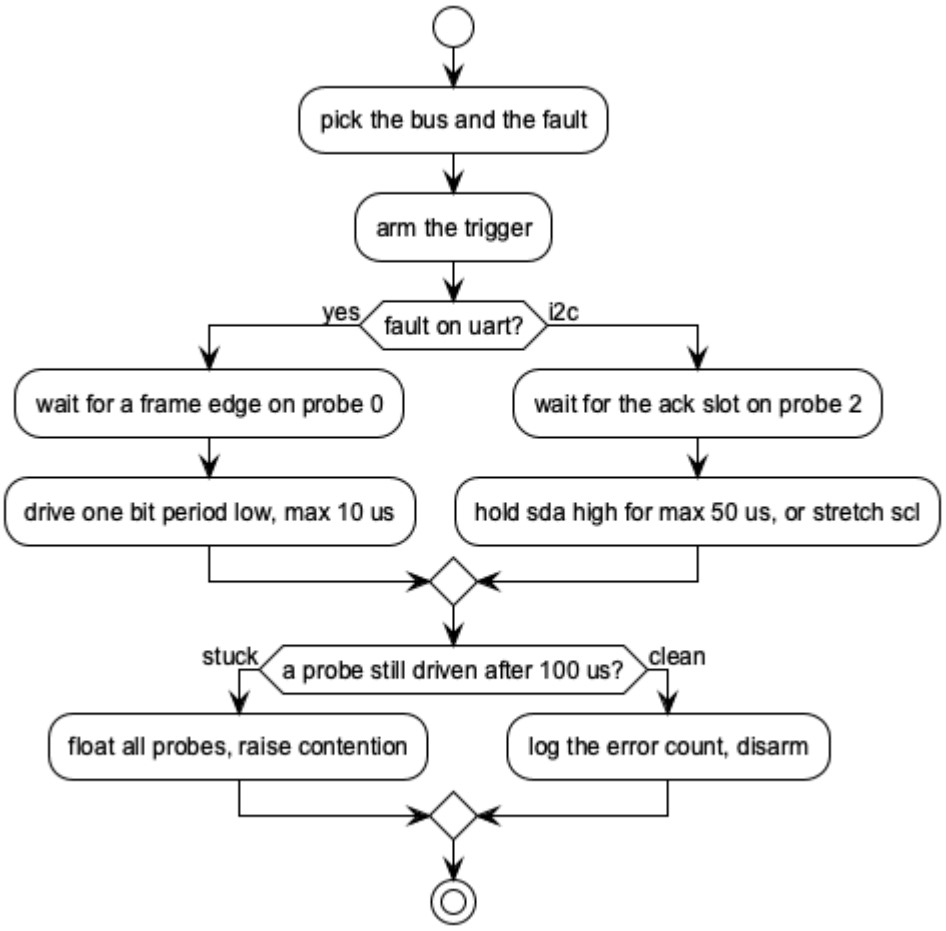


Engine flows

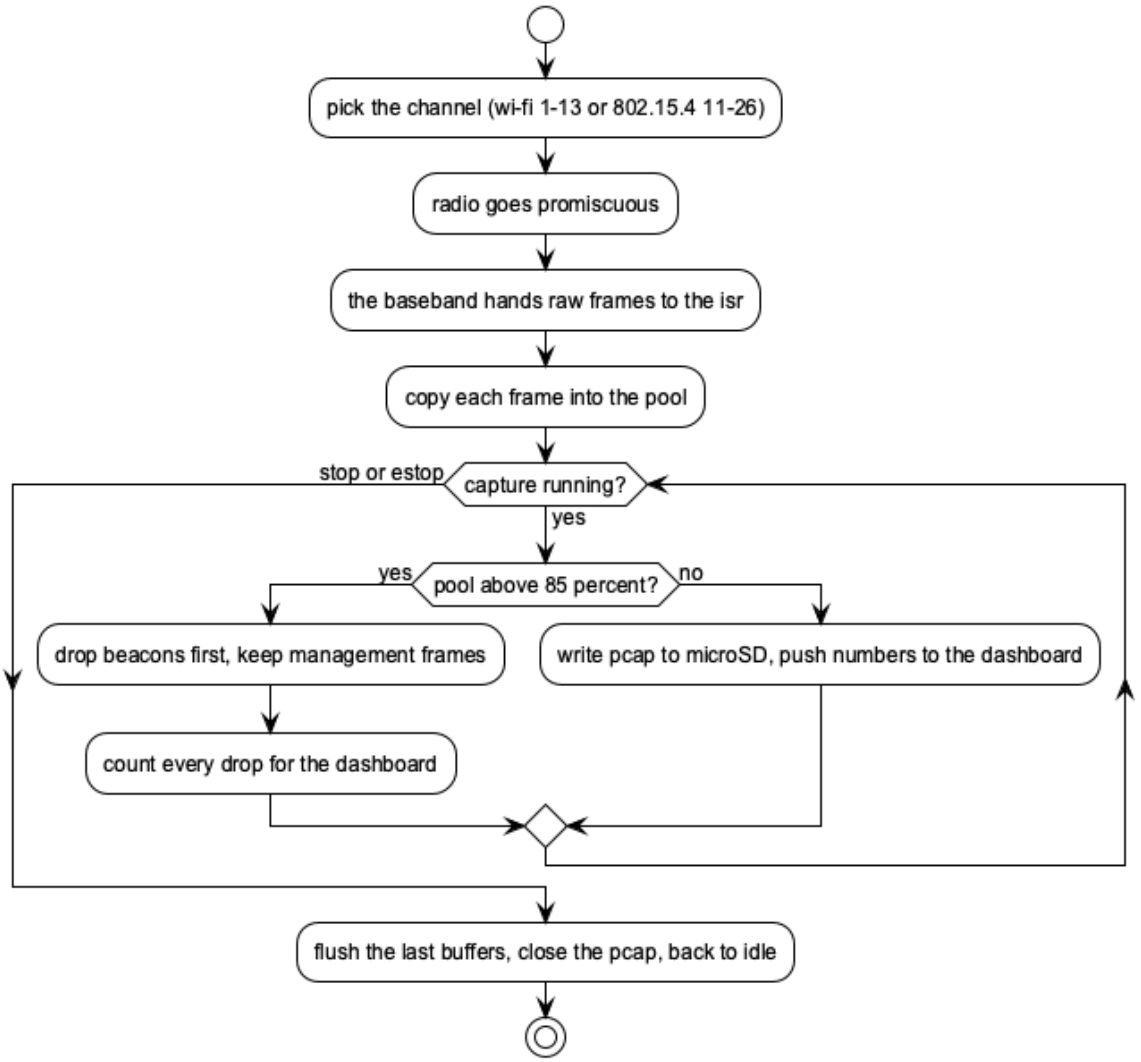
auto discovery:



fault injection:



wireless sniffer:



Invariants

- the e-stop floats every probe from the isr in under 200 ns. It is the boot button, GPIO9, so keep it unpressed for the first 50 ms of any boot. GPIO4 and GPIO5, probes 0 and 1, are strapping pins too, so all probes stay high-Z until boot completes.
- the 220 Ω resistor on probe 0 limits a glitch into a driven high line to 15 mA.
- glitches run one bit period max (10 µs uart, 50 µs i2c) at 5 % duty.
- no malloc while monitoring. everything comes from the pool.
- Maker Pi buttons 1 and 2 (GP20, GP21) are banned, they can short the TCA9554 i2c bus.
- one owner per peripheral per task.

Known gaps

- A0: µT-Kernel with the freertos shim against the closed source wi-fi blobs is still unproven. If it fails, wireless drops to plain FreeRTOS, which is an A-. Gate: a 30 min promiscuous capture with no panic. Tan, Zayne.

- A4: microSD erase stalls up to 40 ms against a 32 KB pool. Ryan.
- A5: discovery can false-lock on bus noise. It needs 16 consecutive valid edge rhythms before `FLG_PROTO_LOCKED`. Tan.
- A6: `esp_wifi_80211_tx` may filter our deauth. If it does, wireless stress falls back to 802.15.4 injection. Zayne, Annie.
- A7: the strapping pins among our GPIOs (GPIO4, GPIO5 on the probes, GPIO9 on the e-stop) must idle at boot. Tan, Joseph.
- A8: the C6 has 4 RMT channels but only 2 support RX, so all four probes cannot be edge-captured by RMT at the same time. Options: run 2 probes on RMT RX and time-share, or move the rest onto a different capture path. Resolved at the week 10 check-in. Tan.

Setup

`./setup.sh` or `./setup.sh --skip-esp-idf`. Reference repos land in `third_party`, which is gitignored. The capture card is a SanDisk Extreme class 10 U3; cheaper cards spike past the 40 ms stall budget.